One Pigeonhole for 3-SAT: A Determinant That Removes the Correction

Siyuan Zhuang

October 31, 2026

Abstract

When parity is replaced by a low-degree polynomial, a low-rank correction appears, and proofs along this route must pay for it. We show that a determinant removes this correction, and we obtain a short proof that the Exponential Time Hypothesis fails: for each $\varepsilon > 0$, 3-SAT with n variables is decided deterministically in time $C_\varepsilon |\varphi|^{O_\varepsilon(1)} 2^{\varepsilon n}$, where $|\varphi|$ is the size of the input. OpenAI and Claude (Anthropic) first announced this statement; our route is different. It has one pigeonhole in the decomposition, and no recursion beyond sparsification. The algorithm never factors the correction and enumerates no value on it: the proof bounds the dimension of its rows, and the algorithm evaluates one explicit formula.

The identity is one line. If all rows of $A - D$ lie in a space of dimension at most r, then expanding $\det D = \det(A - (A - D))$ by rows stops after r rows, because a determinant with more than r rows in that space is 0. Applied to the product of all equalities between the two copies of a bit, it replaces comparisons that flip all digits of a word by comparisons that flip few digits, in all but few rows. Then some digit is flipped by few rows, and the determinant factors across that digit once these rows are named. So the proof rests on two facts: parity is low-degree in commuting swaps except on few eigenvectors, and a determinant vanishes on dependent rows. For two wires and two digits the same matrices give the single-bit crossing identity recorded by Claude, where the low-rank term remains as one product. We also show that no such expansion can keep all its comparisons within fewer than $Q/2$ of the Q digits. The theorem is formalized in Lean 4 down to a multitape machine, with its time and space, for a version of the algorithm whose program holds the small determinants in a table.

1 The result and the strategy

Theorem 1.1 (3-SAT)**.** *For each $\varepsilon > 0$, 3-SAT with n variables is decided deterministically in time $C_\varepsilon |\varphi|^{O_\varepsilon(1)} 2^{\varepsilon n}$ and space $|\varphi|^{O_\varepsilon(1)}$, where $|\varphi|$ is the size of the input φ. On formulas with at most Δ occurrences per variable, the algorithm also computes the number of satisfying assignments, with constants that also depend on Δ.*

The Exponential Time Hypothesis of Impagliazzo and Paturi [IP01] says that for some $\delta > 0$, no algorithm decides 3-SAT on n variables in time $2^{\delta n}$ times a polynomial in the input size. Theorem 1.1 contradicts this: take $\varepsilon < \delta$. After duplicates are removed, the input has size polynomial in n.

1.1 Background and what is simplified

OpenAI [OAI26] and, independently, Claude (Anthropic) [Cla26] announced algorithms for counting with explicit subexponential rates. O'Donnell [O'D26] and Kintali [Kin26] gave expositions of

OpenAI's approach, and a community examination of the argument has been called for [ETH26]. Like Kintali, we fix ε and leave the rate unoptimized. We aim for a proof that is short to state and easy to follow.

In the parity approach, each variable occurrence sits at a pair of words of Q digits, and the two copies of its bit are compared across the transpose of the pair, which exchanges the letters in all digits. Call a comparison across an exchange of some of the digits a *read*. The transpose is replaced by a low-degree polynomial in reads that flip few digits. The two differ by a matrix of low rank, which comes from the few common eigenvectors of the exchanges where the polynomial fails, and the proof must then treat this matrix, the correction. Table 1 lists how. Here a truncated expansion of a determinant (Theorem 1.2) removes the need to factor the correction: the formula keeps only determinants in which few rows come from the difference of the two matrices, and the algorithm enumerates no value on those eigenvectors. A pigeonhole on exponent vectors replaces Hermite recovery; its condition is the degree condition of Kintali's Hermite interpolation. Coefficients are still extracted, from polynomials with at most $(M+1)^Q$ monomials, where M is the number of occurrences. No measurement, no color-count program, and no recursion on cliques or wires is used.

Work	**The low-rank correction**	**Recovery of the value**
OpenAI [OAI26], main text	enumerate integer measurements	Hermite interpolation at nodes
OpenAI [OAI26], Appendix D	modes between groups, recursion on wires	tapered Hermite recovery
Kintali [Kin26]	enumerate integer measurements	Hermite interpolation at nodes
O'Donnell [O'D26]	dynamic program on color counts	Hermite interpolation at nodes
Claude [Cla26]	no such matrix: crossings, recursion on cliques	sum over the recursion
This paper	truncated expansion of a determinant	pigeonhole on exponents

Table 1: Where the proofs differ. For Appendix D of [OAI26] we read the overview and the headings.

The proof keeps the orthogonal characters of the cube and the parity interpolant applied to commuting swaps, from [OAI26], after Alman and Williams [AW15, AW17]; the auxiliary coordinates and swaps that do not depend on the input formula, from [OAI26, Kin26]; and the picture of their orbits as cubes, and a labeling that differs in every digit, from [O'D26]. These facts are used only to show that a set of rows has low dimension. The closest route is O'Donnell's. He writes a matrix polynomial whose value at each of several interpolation points is a block matrix plus the same low-rank E (Theorem 5.1 and Eq. (4.9) there), and he computes the permanent of such a matrix of order n in time $2^{O(s)}n^{O(\operatorname{rank} E)}$ for blocks of size s, up to a polynomial factor (Lemma 2.1 there), with no measurements. He factors E and interpolates at several points, and his route also gives a uniform rate for the sparse permanent. Our expansion is the determinant form of his addition formula (2.2), once a Laplace expansion collects the columns of the rows taken from E. What it adds is the vanishing: a determinant with too many rows in a small space is 0, and a permanent is not. So no factorization of E is needed. The sets T of rows are what remains of the price, and they are enumerated. The expansion itself is one line from multilinearity. What we claim as new is its use here: we did not find it used this way in the works cited in Table 1, or in web searches on determinant expansions and low-rank perturbations.

1.2 The strategy

The obstacle. Give each variable and each clause a word of Q digits. A variable occurrence is a *wire* that compares a bit at the word of the variable with a bit at the word of the clause. Group the words by digit t. The sum over assignments would split if no wire crossed the groups. But the word of a clause differs from the words of its variables in every digit, so each wire crosses the groups of every digit, and no fixed digit helps.

Step 1: reads that flip few digits. A wire lives on a *cell*, a pair of words. Let τ_E exchange the letters of the two words in the digits of E, so a wire reads across all digits, by $\tau_{[Q]}$. A read with $t \notin E$ stays inside a group of digit t. A low-degree polynomial in the commuting swaps agrees with $\tau_{[Q]}$ except on few eigenvectors. So short reads differ from the wires by a low-rank error (Section 3). By Proposition 8.1 the error cannot be zero.

Step 2: trading. Put the wires into a diagonal matrix D, and the short reads into a matrix A. Then $\det D$ is the product of all wires, and the rows of $A - D$ have low dimension (Section 4). The next identity removes the error.

Theorem 1.2 (Trading)**.** *Let D, A be $\nu \times \nu$ matrices over a field, with all rows of $A - D$ in a subspace of dimension at most r. For a set T of rows, let $\langle T \rangle$ have the rows in T from $A - D$ and the other rows from A. Then*

$$\det D = \sum_{|T| \le r} (-1)^{|T|} \det\langle T \rangle.$$

Proof. Expand $\det D = \det(A - (A - D))$ by linearity in the rows. This gives the sum over all sets T. If $|T| > r$, then more than r rows of $\langle T \rangle$ lie in a space of dimension at most r, so $\det\langle T \rangle = 0$. □

So at most r rows of a term contain a read that flips all digits; we call it a *long* read.

Step 3: one pigeonhole. Attach a formal variable z_i to digit i of each read. A term has at most M rows of degree at most R and at most r rows of degree Q. So its total degree is small, and some digit t has a low exponent. Only few rows read across that digit (Section 4).

Step 4: the determinant across a digit. Once the set S of those rows is named, the determinant splits by the letter at digit t (Section 5). A row of S reads the other group, except that its own bit x_i is in the wrong one. Naming the values of those bits fixes this. Then the sum over assignments is a product of sums over the groups (Section 7).

2 The formula

Fix $\Delta \ge 3$ and an integer $m \ge \max(60, \Delta)$, and put $Q = m^4$, $R = m^3$, $L = 6Q$, $K = L^Q$. Take a formula with at most three literals per clause, n variables and at most Δ occurrences per variable, so with $M \le \Delta n$ occurrences. Put $r = \Delta n/m$. We count its solutions. If $n < K^2$, enumerate.

Each occurrence j has a bit x_j, which is 1 if the literal of j is false, and a free bit y_j. Then

$$\#\text{solutions} = \sum_{\text{variables}} \sum_{y} \prod_{e} \Big(1 - \prod_{j \in e} y_j\Big) \prod_{j} \mathbf{1}[x_j = y_j]. \tag{2.1}$$

Words. Words are $[L]^Q$. Give the variables words $s(v)$ cyclically. Since $n \ge K^2$, a word has at most $2n/K$ variables and $2\Delta n/K$ occurrences. Let the *load* of (u, w) be the number of occurrences placed so far of variables with the word u in clauses with the word w. For fixed u, the

loads sum over w to at most $2\Delta n/K$. Going through the clauses, give the clause e a word $o(e)$ that differs in every digit from the words of its variables; at least $(L-3)^Q \geq K(1-3Q/L) = K/2$ words do. Among these, take the word w with the least sum of $\mathrm{load}(s(v), w)$ over the variables v of e. This sum has average at most $3(2\Delta n/K)/(K/2) = 12\Delta n/K^2$, and e adds at most $3 \leq \Delta n/K^2$ to a load. So all loads are at most $h = \lceil 13\Delta n/K^2 \rceil$.

Cells. A *cell* is (u, w, λ) with words u, w that differ in every digit and $\lambda \in [h]$. There are $N \leq K^2 h \leq 14\Delta n$ cells. Since loads are at most h, each occurrence j of v in e gets its own cell $c_j = (s(v), o(e), \lambda)$. Put $d_j = (o(e), s(v), \lambda)$.

3 The window

For $E \subseteq [Q]$, let τ_E exchange the letters of the two words of a cell in the digits of E. So $\tau_{[Q]} c_j = d_j$ and $\tau_E \tau_F = \tau_{E+F}$, with $E + F$ the symmetric difference. Since the two words differ in every digit, the cells $\tau_E c$ for different E are different.

Lemma 3.1. *There are rationals κ_E, $|E| \leq R$, and functions α_l, f_l on cells, with at most N/m^2 indices l, such that for all cells c, d*

$$\sum_{|E| \leq R} \kappa_E \mathbf{1}[\tau_E c = d] - \mathbf{1}[\tau_{[Q]} c = d] = \sum_l \alpha_l(c) f_l(d).$$

Proof. Put $b = 2^Q$. For $\sigma \in \{\pm 1\}^Q$ put $\sigma^E = \prod_{t \in E} \sigma_t$. Then $\sigma^E \sigma^F = \sigma^{E+F}$, and $\sum_\sigma \sigma^E$ is b if $E = \varnothing$ and 0 otherwise. If σ has k minus signs, then $\sigma^{[Q]} = (-1)^k$ and $\sum_t \sigma_t = Q - 2k$. Let p be a polynomial of degree at most R with $p(k) = (-1)^k$ on the at most $R+1$ integers k with $|Q - 2k| \leq R$. Put $k = \sum_t (1 - \sigma_t)/2$ in p, expand, and use $\sigma_t^2 = 1$. This gives κ_E such that $\rho_\sigma := \sum_{|E| \leq R} \kappa_E \sigma^E - \sigma^{[Q]}$ is 0 unless $|\sum_t \sigma_t| > R$. Call such σ *exceptional*. Since $\sum_\sigma (\sum_t \sigma_t)^2 = bQ$, at most $bQ/R^2 = b/m^2$ vectors σ are exceptional.

The *cube* of a cell c_0 is the set of the b cells $\tau_F c_0$. The cubes partition the cells, so there are N/b cubes. Choose a base cell c_0 in each cube. If $c = \tau_F c_0$ and $d = \tau_{F'} c_0$, then for every E we have $\mathbf{1}[\tau_E c = d] = \mathbf{1}[E + F + F' = \varnothing] = b^{-1} \sum_\sigma \sigma^E \sigma^F \sigma^{F'}$. So the left side of the lemma is $b^{-1} \sum_\sigma \rho_\sigma \sigma^F \sigma^{F'}$, a sum over exceptional σ. If c, d lie in different cubes, the left side is 0. So take as indices the pairs l of a cube and an exceptional σ, with $\alpha_l(\tau_F c_0) = \rho_\sigma \sigma^F / b$ and $f_l(\tau_{F'} c_0) = \sigma^{F'}$ on this cube, and both functions 0 outside it. There are at most $(N/b)(b/m^2) = N/m^2$ indices. □

4 Trading and the pigeonhole

Fix values of all bits. Let A, D be the $M \times M$ matrices with

$$A_{ij} = \sum_{|E| \leq R} \kappa_E \mathbf{1}[\tau_E c_i = d_j]\, \mathbf{1}[x_i = y_j], \qquad D_{ij} = \mathbf{1}[\tau_{[Q]} c_i = d_j]\, \mathbf{1}[x_i = y_j].$$

Since $\tau_{[Q]} c_i = d_i$ and the cells d_j are different, D is diagonal and $\det D = \prod_j \mathbf{1}[x_j = y_j]$. By Lemma 3.1, $(A - D)_{ij} = \sum_l \alpha_l(c_i) f_l(d_j) \mathbf{1}[x_i = y_j]$. So every row of $A - D$ lies in the span of the at most $2N/m^2$ vectors $(f_l(d_j)\mathbf{1}[\beta = y_j])_j$, $\beta \in \{0, 1\}$, and $2N/m^2 \leq 28\Delta n/m^2 \leq r$.

By Theorem 1.2,

$$\prod_j \mathbf{1}[x_j = y_j] = \sum_{|T| \leq r} (-1)^{|T|} \det\langle T \rangle. \tag{4.1}$$

Fix T. An entry (i,j) of $\langle T\rangle$ can be nonzero only if $\tau_E c_i = d_j$ for some E; this E is unique, and we call it E_{ij}. In a row outside T, a nonzero entry has $|E_{ij}| \le R$. In a row in T, it has $|E_{ij}| \le R$ or $E_{ij} = [Q]$. Let $z_1, \dots, z_Q$ be formal variables, and let $\langle T\rangle_z$ be $\langle T\rangle$ with each entry (i,j) multiplied by $\prod_{t\in E_{ij}} z_t$. In the expansion of $\det\langle T\rangle_z$ over permutations, a permutation π gives a monomial z^a in which a_t is the number of rows i with $t \in E_{i\pi(i)}$. So

$$\sum_t a_t = \sum_i |E_{i\pi(i)}| \le MR + |T|Q \le \Delta n R + rQ = 2rQ,$$

since $R = Q/m$, and some $a_t \le 2r$. For a polynomial f in z, let $\mathrm{Sum}_t(f)$ be the sum of its coefficients at the monomials z^a with $a_t \le 2r$, and $a_{t'} > 2r$ for $t' < t$. Each monomial of $\det\langle T\rangle_z$ is counted for exactly one t, and the sum of all its coefficients is $\det\langle T\rangle$. So $\det\langle T\rangle = \sum_t \mathrm{Sum}_t(\det\langle T\rangle_z)$.

Fix t. For a set S of rows, let $H_{T,t,S}$ be $\langle T\rangle_z$ with only these entries kept: in the rows of S those with $t \in E_{ij}$, and in the other rows those with $t \notin E_{ij}$. By linearity in the rows, $\det\langle T\rangle_z = \sum_S \det H_{T,t,S}$. Each monomial of $\det H_{T,t,S}$ has $a_t = |S|$, so $\mathrm{Sum}_t(\det H_{T,t,S}) = 0$ for $|S| > 2r$. Hence

$$\prod_j \mathbf{1}[x_j = y_j] = \sum_{|T|\le r} (-1)^{|T|} \sum_t \sum_{|S|\le 2r} \mathrm{Sum}_t(\det H_{T,t,S}). \tag{4.2}$$

5 The determinant across a digit

Equation (4.2) holds for all values of the bits. Now let the bits vary. Fix T, t, S and $H = H_{T,t,S}$. Then $H_{ij} = \gamma_{ij}\mathbf{1}[x_i = y_j]$, where γ_{ij} is zero or a rational times a monomial in z, and does not depend on the bits. For an occurrence i of v in e, let s_i, o_i be the letters of $s(v), o(e)$ at digit t. If $\gamma_{ij} \ne 0$, comparing first words at digit t in $\tau_{E_{ij}} c_i = d_j$ gives $o_j = s_i$ if $t \notin E_{ij}$, and $o_j = o_i$ if $t \in E_{ij}$; and $t \in E_{ij}$ exactly when $i \in S$. So with $g_i = s_i$ for $i \notin S$, and $g_i = o_i$ for $i \in S$,

$$\gamma_{ij} \ne 0 \quad\Longrightarrow\quad o_j = g_i. \tag{5.1}$$

The *bits of a letter* g are the variables v with $s(v)_t = g$ and the bits y_j with $o_j = g$. Each bit belongs to exactly one letter. For $\xi \in \{0,1\}^S$, let H^ξ be H with x_i replaced by the number ξ_i for $i \in S$. Then $\det H = \sum_\xi \det H^\xi \prod_{i\in S} \mathbf{1}[x_i = \xi_i]$.

Let H^ξ_g be the submatrix of H^ξ with the rows that have $g_i = g$ and the columns that have $o_j = g$. By (5.1), all entries of H^ξ outside these submatrices are 0. An entry of H^ξ_g reads y_j with $o_j = g$ and, if $i \notin S$, reads x_i with $s_i = g$, so it reads only bits of g. Call S *balanced* if every H^ξ_g is square; this depends only on t, S. There are M rows and M columns, so if S is not balanced, some H^ξ_g has more rows than columns. These rows are dependent, and $\det H^\xi = 0$. If S is balanced, listing the rows and the columns by letter makes H^ξ block diagonal, so $\det H^\xi = \mathrm{sgn}(t,S)\prod_g \det H^\xi_g$, with a sign that depends only on t, S. Put

$$\Psi_g = \prod_{i\in S:\, s_i = g} \mathbf{1}[x_i = \xi_i] \prod_{e:\, o(e)_t = g} \Big(1 - \prod_{j\in e} y_j\Big) \det H^\xi_g,$$

a polynomial in z whose coefficients depend only on the bits of g. Over all g, each indicator and each clause factor appears exactly once. So the sum of $\prod_g \Psi_g$ over all bits equals $\prod_g \sum_{\text{bits of } g} \Psi_g$. With (2.1), (4.2) and the linearity of Sum_t, this gives

$$\#\text{solutions} = \sum_{|T|\le r} (-1)^{|T|} \sum_t \sum_{\substack{|S|\le 2r \\ S \text{ balanced}}} \mathrm{sgn}(t,S) \sum_{\xi\in\{0,1\}^S} \mathrm{Sum}_t\Big(\prod_g \sum_{\text{bits of } g} \Psi_g\Big). \tag{5.2}$$

6 A worked example

We follow the identities on one square, with two letters, $Q = 2$ and $R = 1$, and with one occurrence on each cell; in this section only, the parameters have these values, and there are no clauses. These parameters are far below those of the proof: the example illustrates the identities and shows no saving. The script `checks/check_example.py` in the directory of this paper checks by exact arithmetic the values below, the identity on all 256 bit points, and formula (4.2) with the factorization by letters on all 256 bit points.

The cells. A cell is determined by its first word $u \in \{00, 01, 10, 11\}$, since its second word is the complement $\bar{u}$. The swap $\tau_{\{i\}}$ flips digit i of u, and $\tau_{[Q]}$ is the antipode. Index the occurrences by u, so c_u has first word u, and d_u has first word $\bar{u}$.

The window. Here $p(k) = 1 - 2k$ has $p(k) = (-1)^k$ at $k = 0, 1$. Substituting $k = (2 - \sigma_1 - \sigma_2)/2$ gives $p = \sigma_1 + \sigma_2 - 1$. So $\kappa_\varnothing = -1$ and $\kappa_{\{1\}} = \kappa_{\{2\}} = 1$. Only $\sigma = (-1, -1)$ has $\rho_\sigma \neq 0$, with $\rho_\sigma = -4$, and the difference of Lemma 3.1 is $-\mathbf{v}\mathbf{v}^\mathsf{T}$ for a vector $\mathbf{v}$ with entries ± 1, of rank 1. So $r = 2$.

The identity. Write $e_{uu'} = \mathbf{1}[x_u = y_{u'}]$. The entry of A at the row u and the column u' is $\kappa_E e_{uu'}$, for the E with $\tau_E c_u = d_{u'}$. In the order $00, 01, 10, 11$, this E is $\{1, 2\}$ on the diagonal, where A is 0, and it is $\varnothing$ on the antidiagonal, where the coefficient is -1. So

$$A = \begin{pmatrix} 0 & e_{00,01} & e_{00,10} & -e_{00,11} \\ e_{01,00} & 0 & -e_{01,10} & e_{01,11} \\ e_{10,00} & -e_{10,01} & 0 & e_{10,11} \\ -e_{11,00} & e_{11,01} & e_{11,10} & 0 \end{pmatrix},$$

and D is the diagonal matrix of the e_{uu}. Let $F_k = \sum_{|T|=k} \det\langle T\rangle$. For $x = y = (1, 0, 1, 0)$, we get $F_0 = 1$, $F_1 = 4$, $F_2 = 4$, and $F_0 - F_1 + F_2 = 1 = \det D$. For $x = y = (0, 0, 0, 0)$, we get $F_0 = -3$, $F_1 = -4$, $F_2 = 0$, and again the sum is 1. For $x = (1, 0, 0, 0)$ and $y = (0, 0, 0, 0)$, the first row of A and of $A - D$ has only $e_{00,\cdot} = 0$, so all F_k are 0, and $\det D = 0$.

The pigeonhole. The budget is $MR + |T|Q \leq 4 + 2 \cdot 2 = 8$, and $2r = 4$. Each exponent is at most $M = 4$, so $|S| \leq 2r$ removes nothing, every monomial is counted at $t = 1$, and digit 2 contributes nothing. The example is too small for the pigeonhole to act.

The letters. Take $t = 1$ and $S = \{00, 10\}$. The column u' has $o_{u'} = \bar{u}'_1$, so the columns of letter 0 are $\{10, 11\}$, and those of letter 1 are $\{00, 01\}$. The row $01 \notin S$ has $g = s = 0$, and the row $11 \notin S$ has $g = 1$. The row $00 \in S$ has $g = o = 1$, and the row $10 \in S$ has $g = o = 0$. So the rows of letter 0 are $\{01, 10\}$, those of letter 1 are $\{00, 11\}$, and S is balanced. By contrast $S = \{00\}$ is not: letter 1 then has the three rows $00, 10, 11$ and two columns. The first block reads the bit x_{01}, the number ξ_{10}, and the bits y_{10}, y_{11}. These bits belong to letter 0. The indicator $\mathbf{1}[x_{10} = \xi_{10}]$ reads x_{10}, whose letter is $s_{10} = 1$, so it is a factor of Ψ_1.

Two wires: the crossing. The smallest case of these matrices is a known identity. Keep $Q = 2$, $R = 1$ and the numbers κ_E above, so that $\sum_{|E|\leq 1} \kappa_E \tau_E$ is $\tau_{\{1\}} + \tau_{\{2\}} - 1$, but take only two occurrences, on the cells c_1 and $c_2 = \tau_{\{1\}} c_1$, so that A, D have order 2. Then $\tau_E c_1 = d_2$ and $\tau_E c_2 = d_1$ hold for a single flip, and $\tau_E c_i = d_i$ for no short read. So the numbers k_{ij} with $A_{ij} = k_{ij}\mathbf{1}[x_i = y_j]$ form the matrix with rows $(0, 1)$ and $(1, 0)$, and $k - I$ has rank 1. The rows

of $A - D$ span a space of dimension $2 = \nu$ for general bits, so Theorem 1.2 keeps the term with both rows from $A - D$, and that term is

$$\det(A - D) = \mathbf{1}[x_1 = y_1]\,\mathbf{1}[x_2 = y_2] - \mathbf{1}[x_1 = y_2]\,\mathbf{1}[x_2 = y_1] = (x_2 - x_1)(y_2 - y_1), \tag{6.1}$$

on bits. This is the expansion of a crossing in [Cla26, Appendix H]. Its last term is one product across the cut between the bits x and the bits y. So the parity window and this single-bit crossing are one expansion at two sizes. With two wires the term of $A - D$ remains, as one product. With many wires and few exceptional vectors, the dimension of the rows is below the number of rows, and that term vanishes. This relates the window to this single-bit identity only. The recursion on cliques of [Cla26] rests on other decompositions of the crossing, and no proof there uses its Appendix H.

7 The cost and the end of the proof

We evaluate (5.2). Since $M/m \le r$, for every $q > 0$ we have $\sum_{k \le qr} \binom{M}{k} \le m^{qr} \sum_k \binom{M}{k} m^{-k} = m^{qr}(1 + 1/m)^M \le m^{qr} \mathrm{e}^r$. So there are at most $(\mathrm{e}m)^r$ sets T and at most $4^r(\mathrm{e}m^2)^r$ pairs (S, ξ), so at most $(4\mathrm{e}^2 m^3)^r$ triples (T, S, ξ).

A letter g is the letter at digit t of K/L words, so it has at most $2n/L$ variables. At most $K^2 h/L \le 14\Delta n/L$ cells have the letter g at digit t of the first word, and the cells d_j are different, so g has at most $14\Delta n/L$ bits y_j. So g has at most $2^{16\Delta n/L} \le 2^{n/m}$ assignments, since $L = 6m^4$ and $\Delta \le m$.

The numbers κ_E depend only on m. All polynomials have exponents at most M in each z_t, so they have at most $(M + 1)^Q$ monomials, with coefficients of polynomially many bits, and a determinant of such polynomials is found from its values at $(M + 1)^Q$ points. So one Ψ_g, the product over the L letters, one Sum_t, and the words and signs each take time polynomial in n, with degree depending on m, and this is at most $C_m 2^{n/m}$. Multiply by the triples, the Q digits, the L letters and their assignments. Since $2n/m \le r$ and $8\mathrm{e}^2 < 60 \le m$, one formula takes time at most $C_m(8\mathrm{e}^2 m^3)^r \le C_m m^{4r} = C_m 2^{4\Delta n \log_2 m/m}$ times a polynomial in n. A set, a value ξ and a table of at most $(M + 1)^Q$ coefficients each have $n^{O_m(1)}$ bits, and the loops over triples, digits, letters and assignments keep only their current position and a running sum. So the space is $n^{O_m(1)}$.

Few occurrences. The reduction follows the sparsification lemma [IPZ01], in a form that we analyze directly. Fix $\ell \ge 2$. Normalize by deleting tautologies, repeated literals, repeated clauses, and clauses containing another clause. While one of these cases applies, take the first available one, breaking ties lexicographically:

(i) a literal Z_0 lies in at least ℓ binary clauses;

(ii) a pair Z_0 of literals lies in at least ℓ ternary clauses;

(iii) a literal Z_0 lies in at least ℓ^2 ternary clauses.

Write all those clauses as $Z_0 \vee C_i$. Since $\bigwedge_i (Z_0 \vee C_i) \equiv Z_0 \vee \bigwedge_i C_i$, branch: add the clause Z_0, or add all clauses C_i. Then normalize. The formula is satisfiable if and only if some leaf is. Leaves may share solutions, so we only test whether a count is positive. At a leaf, a literal lies in at most $1 + \ell + \ell^2$ clauses, so a variable occurs at most $2(1 + \ell + \ell^2) \le 4\ell^2$ times.

Lemma 7.1. *The tree has depth at most* $10n\ell$ *and at most* $\exp((20\log\ell+10)n/\ell)$ *leaves.*

Proof. Added clauses are new and distinct: the C_i differ, and an existing subclause would have removed a clause $Z_0 \vee C_i$. For the analysis, let ν count unit clauses, and let μ be the largest size of a family of binary clauses in which each literal occurs fewer than 2ℓ times. Counting each clause at its two literals gives $\mu \le 2n\ell$, and $\nu \le 2n$, so $\Upsilon = 4\ell\nu + \mu \le 10n\ell$. Adding s new units keeps the old units and removes at most $2\ell s$ clauses from a largest family, so Υ grows by at least $2\ell s$.

Before (ii) or (iii), every literal lies in fewer than ℓ binary clauses. Case (ii) adds one binary clause. Case (iii) adds fewer than ℓ containing any one literal, else (ii) would apply. So all degrees stay below 2ℓ, and each new binary clause raises Υ by one. In all cases, the two children gain at least 1 and $\eta := \ell^2$. Put $\vartheta = (\log\eta + 1)/\eta$. From $e^{\vartheta} \ge 1 + 1/\eta$ and $e^{\vartheta\eta} = \mathrm{e}\eta \ge 2\eta$ we get $e^{-\vartheta} + e^{-\vartheta\eta} \le \eta/(\eta+1) + 1/(2\eta) \le 1$, so $e^{-\vartheta\Upsilon}$ at a node is at least the sum over its children. Hence the number of leaves is at most $e^{10\vartheta n\ell} = \exp((20\log\ell+10)n/\ell)$, and each step raises Υ, so the depth is at most $10n\ell$. □

Proof of Theorem 1.1. Fix ε. Choose ℓ with $(20\log\ell+10)/\ell \le \varepsilon/4$, so that the number of leaves is at most $\mathrm{e}^{\varepsilon n/4} \le 2^{\varepsilon n/2}$, and put $\Delta = 4\ell^2$. Choose $m \ge \max(60, \Delta)$ with $4\Delta\log_2 m/m \le \varepsilon/2$. Traverse the tree depth first, and accept at the first positive count. Store only the path of the current node, which has at most $10n\ell$ bits by Lemma 7.1, and recompute the formula at the node from the root. A normalized formula has $O(n^3)$ clauses, so a step costs time polynomial in n and $|\varphi|$, and the tree has at most twice as many nodes as leaves. At a leaf, run the algorithm above, which enumerates if $n < K^2$. The time is $C_\varepsilon|\varphi|^{O_\varepsilon(1)}2^{\varepsilon n}$, and the space is $|\varphi|^{O_\varepsilon(1)}$, since $n \le |\varphi|$. For a formula with at most Δ_0 occurrences per variable, skip the branching, and run Sections 2–5 with $\Delta = \max(\Delta_0, 3)$ and with $m \ge \max(60, \Delta)$ such that $4\Delta\log_2 m/m \le \varepsilon$. Then (5.2) is the number of satisfying assignments. □

8 Limits

Three statements explain why the proof has this form. The numbers Q and R are free in this section. Take a cell whose two words differ in every digit. Its cube is a copy of $\{0,1\}^Q$, with 2^Q corners: the swap of digit i flips coordinate i, and $\tau_{[Q]}$ is the antipodal map $a \mapsto \bar{a}$.

No exact short read. The next proposition says that the error in Step 1 of Section 1.2 cannot be removed for free. Let X be a bit vector on the corners.

Proposition 8.1. *Suppose a function* $\Gamma(z, X)$*, polynomial in formal variables* $z_1, \dots, z_Q$*, satisfies* $\Gamma(1, \dots, 1, X) = X_{\bar{a}}$ *for all bit vectors* X *on the corners, and that for each* t*,* $\Gamma(z, X)$ *on* $z_t = 0$ *does not depend on the entry* $X_{\bar{a}}$*, where* a *is a corner. Then* Γ *has total degree at least* Q *in* z*.*

Proof. Let $\Theta(z)$ be the difference of Γ between $X_{\bar{a}} = 1$ and $X_{\bar{a}} = 0$, with the other entries fixed. Then $\Theta(1, \dots, 1) = 1$, and $z_t \mid \Theta$ for each t, so $\prod_t z_t \mid \Theta \ne 0$. □

This extends a root count of O'Donnell [O'D26, Section 5.1] to schemes that need not be linear. For the linear scheme of Section 3 the conclusion is immediate: if $R < Q$ and $d = \tau_{[Q]}c$, then the sum in Lemma 3.1 is 0 and the second term is 1, so the error is not 0. The proposition says the same for every scheme of this form, linear in X or not.

Reads of length above R *are necessary.* Does the product of wires have an expansion in which every read flips at most R digits? On one cube, with one occurrence on each corner, index x by

the corner of c_j and y by the corner of d_j, so that a wire compares x_a with $y_{\bar{a}}$. A *permutation product* of a permutation π of the corners is $\Pi_\pi(x,y) = \prod_a \mathbf{1}[x_a = y_{\pi a}]$. The Leibniz formula writes each determinant in Theorem 1.2 as a combination of these, and the product of the wires is the permutation product of $a \mapsto \bar{a}$. Write $|a \oplus a'|$ for the number of coordinates where a, a' differ, and call $|a \oplus \pi a|$ the *length* of the read of π at a. For a corner a let $s(a) = \sum_i (-1)^{a_i}$.

Proposition 8.2. *Let $Q \geq 1$, and suppose $\prod_a \mathbf{1}[x_a = y_{\bar{a}}]$ is a linear combination of permutation products on the bit points (x, y).*

1. *If $R < Q/2$, then for each corner a_0, the combination has a term with $|a_0 \oplus \pi a_0| > R$.*
2. *The combination has a term with $\sum_a |a \oplus \pi a| \geq \sum_a |s(a)| \geq 2^Q \sqrt{Q/3}$.*

Proof. Both parts evaluate at a bit point with $x_a = \zeta_a$ and $y_a = \zeta_{\bar{a}}$, where the bits ζ take each value at exactly one of $a, \bar{a}$, for all a. At this point, the left side is 1. A term Π_π is nonzero at this point only if $\zeta_a = \zeta_{\overline{\pi a}}$, that is $\zeta_{\pi a} \neq \zeta_a$, for all a.

(1) The Hamming ball of radius R around a_0 contains no antipodal pair, since two of its points differ in at most $2R < Q$ coordinates. So some ζ as above has the value 0 on the whole ball. A nonzero term then has πa_0 outside the ball.

(2) Let $\zeta_a = 0$ if $s(a) > 0$, and $\zeta_a = 1$ if $s(a) < 0$; since $s(\bar{a}) = -s(a)$, this is consistent with the requirement, and on the corners with $s(a) = 0$ choose one of each antipodal pair. For a nonzero term, $s(a)$ and $s(\pi a)$ do not have strictly the same sign. Flipping a coordinate changes s by 2, so $2|a \oplus \pi a| \geq |s(a) - s(\pi a)| = |s(a)| + |s(\pi a)|$. Sum over a, using that π is a bijection: $\sum_a |a \oplus \pi a| \geq \sum_a |s(a)|$. For the uniform corner, $\mathbb{E}s^2 = Q$ and $\mathbb{E}s^4 \leq 3Q^2$, and Hölder gives $\mathbb{E}s^2 \leq (\mathbb{E}|s|)^{2/3} (\mathbb{E}s^4)^{1/3}$, so $\mathbb{E}|s| \geq \sqrt{Q/3}$. □

Part (2) is attained for $Q = 2$: by the script of Section 6, the product lies in the span of the permutation products of total length at most $4 = \sum_a |s(a)|$ on all bit points, and not in the span of those of length at most 2. In the form of Theorem 1.2, a term with k rows of length above R and all other rows of length at most R on a cube has total length at most $(2^Q - k)R + kQ$. So if $R < \sqrt{Q/3}$, then some term has $k \geq 2^Q(\sqrt{Q/3} - R)/(Q - R)$ such rows on that cube. At the parameters of Section 2 we have $R = m^3 > \sqrt{Q}$, so this count forces nothing, and part (1) is what requires reads of length above R. Part (2) also suggests a limit of the pigeonhole, though we do not prove one for all orbits: the length of a read on a cube is its degree in the variables z of Section 4, so on a cube some term has average exponent at least $2^Q/\sqrt{3Q}$ over the digits. That is the order of the next bound.

The number of products. The next bound holds for every sum of products over the partitions P_t defined below. It concerns the equality of a matrix with its transpose, which differs from the product of wires in (5.2): it runs over all pairs of words, while the cells of Section 2 have words that differ in every digit and an index λ. Also, a term of (5.2) is a product over the letters of digit t only after Sum_t is expanded into monomials. So we use the bound only as a comparison. For $X, Y \in \{0,1\}^{[G]^Q \times [G]^Q}$, with $\mathcal{N} = G^{2Q}$ entries each, let $\mathbf{T}(X,Y) = \mathbf{1}[Y_{w,u} = X_{u,w}\ \forall u, w]$. For a digit t, the partition P_t sorts the entries by digit t of their row, and a *product over P_t* is a product of functions of its blocks. Fix t and a coloring $\chi : [G]^Q \to \{\pm 1\}$ of the words that depends only on digit t. Each block of P_t then lies on one side of χ, so merging the blocks writes a product over P_t as a function of the entries with $\chi = 1$ times a function of the entries with $\chi = -1$, by the color of the row. We compare ranks across the cut between all X bits and all Y bits. Fix a set W_0 of pairs of words. Set to 0 every $Y_{u,w}$ and every $X_{w,u}$ with $(u,w) \notin W_0$. Then $\mathbf{T}$ becomes the identity matrix between $(X_{w,u})_{(u,w)\in W_0}$ and $(Y_{u,w})_{(u,w)\in W_0}$, of rank $2^{|W_0|}$. The

bit $Y_{u,w}$ lies on the side of u, and $X_{w,u}$ lies on the side of w. Let n^+ count the $(u,w) \in W_0$ with $\chi(u) = 1$, $\chi(w) = -1$, let n^- count those with $\chi(u) = -1$, $\chi(w) = 1$, and let $n^0 = |W_0| - n^+ - n^-$, split as $n' + n''$ according to the side of the two words. The side $\chi = 1$ holds $n^- + n'$ bits of X and $n^+ + n'$ bits of Y, and the side $\chi = -1$ holds $n^+ + n''$ bits of X and $n^- + n''$ bits of Y. So the rank between the X and Y bits of a function of one side is at most $2^{\min(n^+,n^-)+n'}$ or $2^{\min(n^+,n^-)+n''}$. So one product has rank at most

$$2^{|W_0|-|n^+-n^-|}. \tag{8.1}$$

Proposition 8.3. *Let G be even. If $\mathbf{T} = \mathbf{T}_1 + \cdots + \mathbf{T}_Q$ on bit matrices, where $\mathbf{T}_t$ is a linear combination of r_t products over P_t, then*

$$\sum_t r_t \ge 2^{D^*}, \qquad D^* = \frac{\mathcal{N}}{2}\frac{\binom{2Q}{Q}}{4^Q} \ge \frac{\mathcal{N}}{4\sqrt{Q}}.$$

Proof. Put $\chi_t(u) = 1$ if $u_t \le G/2$ and $\chi_t(u) = -1$ otherwise. Let $s'(u) = \sum_t \chi_t(u)$ and $W_0 = \{(u,w) : s'(u) > s'(w)\}$. Permuting the digits fixes W_0, so $D^* := n_t^+ - n_t^-$ does not depend on t. Summing over t, and using $\chi_t(u) - \chi_t(w) \in \{0, \pm 2\}$,

$$QD^* = \sum_{(u,w)\in W_0} \frac{s'(u) - s'(w)}{2} = \frac{\mathcal{N}}{4}\,\mathbb{E}|s'(u) - s'(w)|$$

for uniform u, w, because the sum over W_0 takes half of the pairs with $s'(u) \ne s'(w)$. Here $s'(u) - s'(w)$ has the law of a sum of $2Q$ independent signs, whose expected absolute value is $2Q\binom{2Q}{Q}4^{-Q}$. This gives the formula for D^*. The bound $\binom{2Q}{Q} \ge 4^Q/(2\sqrt{Q})$ gives $D^* \ge \mathcal{N}/(4\sqrt{Q})$. Comparing ranks by (8.1), $2^{|W_0|} \le \sum_t r_t 2^{|W_0|-D^*}$. □

So a sum of products over the partitions P_t that equals $\mathbf{T}$ has an exponent of order at least $\mathcal{N}/\sqrt{Q}$. With $Q = m^4$, the count of Section 7 has an exponent of order $N \log m/m = N(\log Q)/Q^{1/4}$, up to constants, where N is the number of cells of Section 2. The two exponents differ by a factor of about $Q^{1/4} \log Q$, although the two products are not the same.

9 Formalization

The Lean 4 development [MU21, mC20] is in the directory `lean/` of the repository `https://github.com/suquark/NotETHSimplified`. The proof of this paper is the library `OnePigeonhole`. It uses the libraries `TransposeBase` and `SatBase` for the word language, the network of a formula, the branching tree and the machine, and `FormalizationKit` for resource bounds on the machine. The file `OnePigeonhole/Audit.lean` reports that the theorems listed there depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`, or on a subset of them, and the library contains no `sorry`. The file `lean/Audit.lean` reports the same for `exact_degree`, `cut_prop` and `digit_cost`. The directory `lean/third_party/` holds OpenAI's machine with its license and a patch record for Lean 4.33.1. Table 2 maps the paper to the Lean declarations.

The Lean proof departs from the text in these ways, and the claims are unchanged except where stated. Lemma 3.1 is stated for the wires, and κ_E depends only on $|E|$. The load bound is $h = \lfloor 6(\lfloor n/K \rfloor + 1)\Delta/K \rfloor + 3$, and Lean proves $K^2 h \le 13\Delta n$ (`sq_mul_capB_le`). Equation (2.1) is in network form: `TransposeBase.Sat.net_count` has one bit per wire, `glue_eqv` gives the

Paper	Lean declaration	Module of `OnePigeonhole`
Words and cells	`load_table_le`, `sq_mul_capB_le`, `wiresOf`, `onNodes_wiresOf`	`Greedy`
Lemma 3.1	`card_exc_mul_sq_le`, `kappa`, `Bmat_sub_one_eq`, `rank_Bmat_sub_one_le`	`Walsh`, `Window`
Theorem 1.2, Eq. (4.1)	`det_eq_sum_rowMix`, `prod_eqv_eq_sum_rowMix`	`Trade`
Eq. (4.2)	`sum_exp_le`, `prod_eqv_eq_sum_sumT`, `prod_eqv_eq_grid`	`Pigeon`
Eq. (5.1), blocks	`letter_of_coefH_ne_zero`, `det_Hz_eq_prod`, `det_cubeMat`	`Blocks`, `Cube`
Eq. (5.2)	`det_decomp_sumT`, `net_decomp_sumT`, `det_decomp_masks`, `net_decomp_masks`	`Theorem`, `Net`
Word program, 3-SAT	`Pseudo.Final.pseudo_main`	`Pseudo.Final`
Machine, ETH	`Machine.machine_main`, `Machine.not_ETH`	`Machine.FinalMain`
Lemma 7.1	`sparseTree_leaves`, `sparseTree_height`	in `SatBase`
Compiler to stacks	`compile_correct`	in `TransposeBase.Machine.Compile`
Proposition 8.1	`exact_degree`	in `TransposeBase.Exact`
Proposition 8.3	`cut_prop`, `digit_cost`	in `TransposeBase.Exact`, `.Moments`

Table 2: Map from the paper to the Lean development.

two copies of each bit, and each variable has one more wire, to a node of its own. So a letter has at most $2K^2h/L$ bits, not $16\Delta n/L$. The parameters are $r = \lfloor M/m \rfloor$ and $J = 2\lfloor M/m \rfloor + 1$, with $80 \le m$ and $26(1+\Delta) \le m$, where M includes the wires of the variables. The cost has the factor $(6m)^9 \exp(9(\ln m + 3)M/m)$, and the choice of m is `exists_m`. The program reads the coefficients from values on a grid of $(M+1)^Q$ points (`prod_eqv_eq_grid`); the formal-polynomial forms of (4.2) and (5.2) are also theorems (`prod_eqv_eq_sum_sumT`, `det_decomp_sumT`, `net_decomp_sumT`). The determinant is factored further than by letters: into cubes and the two sides of the digit t (`det_Hz_eq_prod`, `det_cubeMat`), with a permutation `pairOf` whose sign plays the role of $\mathrm{sgn}(t, S)$. In place of the values ξ on S, Lean expands by a set $U \subseteq S$ (`det_Hz_eq_sum_U`). The formula runs over triples of sets of size below J with a test. Unbalanced S is not removed: both sides of the factorization vanish there. The traversal `treeE` is a bounded loop with no early exit, and its value counts the leaves with a positive count. The program enumerates for $n \le K^2$. The constants of Section 7 are computed by hand, and Lean has its own.

The algorithm. The word language has addition, subtraction and multiplication modulo 2^{w}, division, remainder, powers of 2, comparison, and bounded loops. Its semantics gives the value of a program and the number of word operations, counted over the iterations that actually run. The theorem `Pseudo.Final.pseudo_main` states: for every $\varepsilon > 0$ there are one program P and constants C, b, c such that for every n and every formula F with n variables and clauses of at most three literals, with $\mathsf{w} = c(n+1)^3$ and input the bitset of F and the number n, the value of P is nonzero exactly when F is satisfiable, and the number of word operations is at most $C(n+1)^b 2^{\varepsilon n}$. The program has no determinant code. Since a side of a cube has order at most 2^Q, a constant, the program holds a literal table of the coefficients of those determinants, scaled by one denominator (`Pseudo.sideZ`, `Pseudo.Leaf.exists_lits`).

The size of the table. The table has $2Q \cdot 2^{5 \cdot 2^Q}$ rows, each with $(2^Q+1)^Q$ entries, and the word width holds all of it (`wConst`). With $Q = m^4$, the word constant c grows at least doubly exponentially in m^4. The table buys a program with no determinant code; the mathematics does not need it. These theorems have the form that refuting ETH requires: a machine for each ε. One algorithm with running time $2^{o(n)}$ is a stronger statement, which Lean does not state.

The machine. The compiler `compile_correct` compiles every program of this language to a program on Boolean stacks, with a time of at most $7000(\mathsf{w}+1)^2$ source instructions for each

word operation, and a bound of $7100(\mathtt{w}+1)^2$ bits on every stack at every prefix, independent of the number of iterations. Together with a parser of the input, and a compiler to a primitive multitape machine of OpenAI [OAI26b], whose code is included in `lean/third_party/` with its license, this gives `Machine.machine_main`: for every $\varepsilon > 0$ there are a primitive machine and constants such that, for every n and every formula φ with n variables and clauses of at most three literals, the machine started on the fixed encoding of φ halts with the answer, in at most $C(|\varphi|+1)^d 2^{\varepsilon n}$ steps, where $|\varphi|$ is the length of the encoding; and every prefix of that same run visits at most $C'(|\varphi|+1)^{d'}$ tape cells. The word width there is $\mathtt{w} = c(|\varphi|+1)^4 + |\varphi|$. The theorem `Machine.not_ETH` states the failure of ETH in this model. The file `OnePigeonhole/Audit.lean` checks by `rfl` that these end statements are those of `TransposeBase/Statements.lean`. A meta-program, `OnePigeonhole/ClosureCheck.lean`, computes the dependency closure of `not_ETH`. It fails unless the closure omits `UncrossingWires` and contains `net_decomp_masks` and `prod_eqv_eq_sum_rowMix`.

What is not formalized. The count of satisfying assignments on sparse formulas is not an end theorem of Lean, although `net_decomp_masks` gives its formula for placed networks. Of the two wires of Section 6, the last equality of (6.1) is `TransposeBase.Sector.skein`; its derivation from A and D is not formalized. Proposition 8.2 is not formalized. The theorem `exact_degree` is stated for an abstract family of polynomials, and `cut_prop` uses its own definition of the transpose, so no theorem links them to `OnePigeonhole`. The constant $1/4$ of Proposition 8.3 is in `cut_prop`; `digit_cost` has $1/5$.

Use of AI

The argument and the Lean proofs were developed in interactive sessions with Claude (Anthropic), building on the public manuscripts [OAI26, O'D26, Cla26, Kin26]. The author is responsible for any errors.

References

[AW15] J. Alman and R. Williams. Probabilistic polynomials and Hamming nearest neighbors. In *Proc. 56th IEEE FOCS*, 2015.

[AW17] J. Alman and R. Williams. Probabilistic rank and matrix rigidity. In *Proc. 49th ACM STOC*, 2017.

[Cla26] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Preprint, October 26, 2026. https://github.com/anthropics/formal-math/tree/16a5ce41304fcdaea4e0be7c132e1c361dc26da5/eth.

[ETH26] R. Impagliazzo, R. Paturi, and R. Williams. ETH refutation: examination and discussion. Web site, October 2026. https://ethdiscussion.blog/2026/10/26/eth-refutation-examination-and-discussion/.

[IP01] R. Impagliazzo and R. Paturi. On the complexity of k-SAT. *J. Comput. System Sci.* 62(2):367–375, 2001.

[IPZ01] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *J. Comput. System Sci.* 63(4):512–530, 2001.

[Kin26] S. Kintali. A simplified proof of ETH is false. Manuscript, October 26, 2026. https://shivakintali.github.io/papers/ETH.pdf.

[MU21] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. 28th CADE*, 2021.

[mC20] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN CPP*, 2020.

[OAI26] OpenAI. The Exponential Time Hypothesis is false. Manuscript, October 17, 2026; revised October 25, 2026. https://github.com/openai/math/blob/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf.

[OAI26b] OpenAI. Lean formalization accompanying "The Exponential Time Hypothesis is false". Repository, commit 4eaee87; see `lean/OAI/Algorithms/SubexponentialSAT/SqrtLogSpace/CompileSpace.lean`. https://github.com/openai/math/tree/4eaee87548250b2682affda7d05d947c05554bae/lean.

[O'D26] R. O'Donnell. Some exposition of "The Exponential Time Hypothesis Is False". Manuscript, October 30, 2026. https://www.cs.cmu.edu/~odonnell/some_eth_false_exposition.pdf.